

LAB4 - REPORT

Martí Gallego Giménez u251445
Mateu Soria Waha u251380

1. Introducció

Aquest Lab implementa la regressió lineal amb el Stochastic Gradient Descent (SGD) i datasets estandarditzats. A més de les classes anteriors (Vector, Record i Model), s'han redissenyat les classes relacionades amb datasets i algoritmes, convertint-les en classes abstractes amb múltiples implementacions concretes.

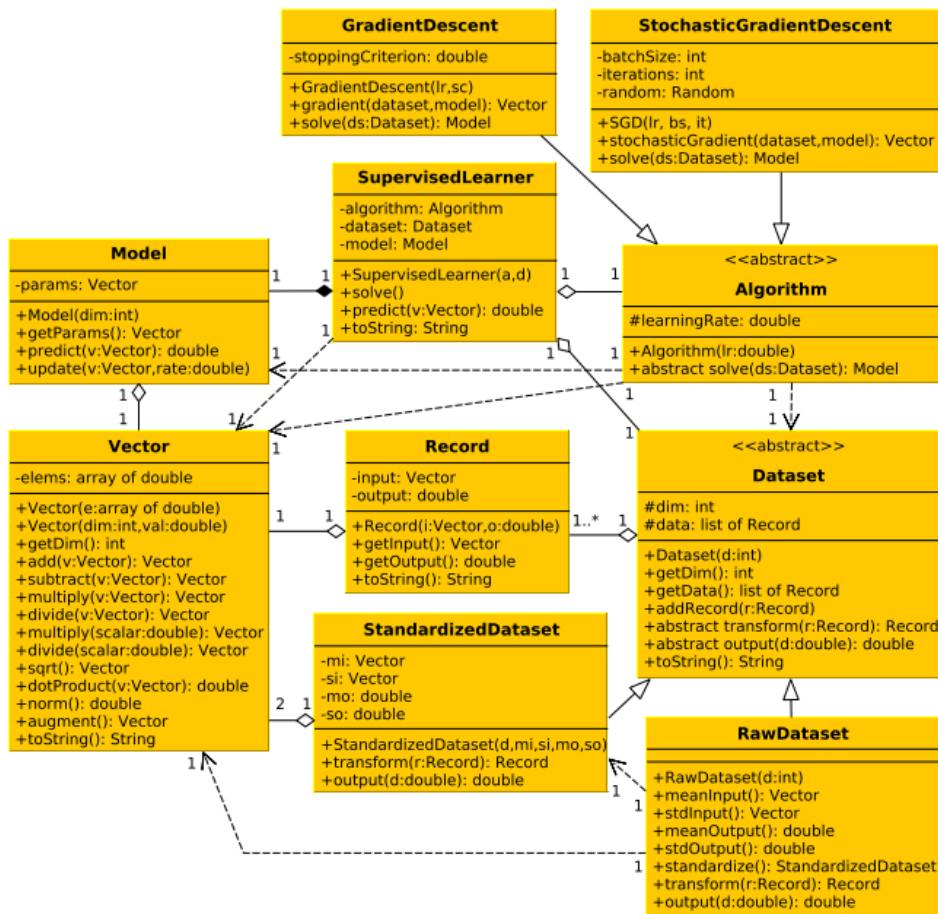
Les principals millores implementades són:

- **Dataset** s'ha convertit en una classe abstracta amb dos mètodes abstractes (transform i output) que permeten gestionar transformacions entre l'espai original i l'espai estandarditzat.
- **RawDataset** hereta de Dataset i conté els mètodes per calcular mitjanes i desviacions estàndard, així com el mètode standardize per crear datasets estandarditzats.
- **StandardizedDataset** hereta de Dataset i implementa les transformacions necessàries per treballar en l'espai estandarditzat i convertir prediccions de nou a l'espai original.
- **Algorithm** s'ha convertit en una classe abstracta que conté la taxa d'aprenentatge compartida per tots els algoritmes i un mètode abstracte solve.
- **GradientDescent** implementa el descens de gradient complet (batch) amb un criteri d'aturada basat en la norma del gradient.
- **StochasticGradientDescent** implementa el descens de gradient estocàstic, que utilitza minibatches aleatoris per actualitzar el model durant un nombre fix d'iteracions.

2. Descripció de la solució

El Lab s'ha implementat dins de la carpeta anomenada Lab4 dins del nostre repositori de [Github](#). Conté, a part dels arxius dels labs anteriors (Vector, Record i Model), els arxius redissenyats Dataset.java (ara abstracta), RawDataset.java, StandardizedDataset.java, Algorithm.java (ara abstracta), GradientDescent.java, StochasticGradientDescent.java, SupervisedLearner.java (modificat) i TestLearner.java, on es proven que totes les implementacions funcionin correctament. Els resultats de les proves es mostren per la terminal.

Les relacions entre les diferents classes són les següents:



2.1. Classe Dataset

Aquesta classe abstracta representa la base per a tots els tipus de datasets (RawDataset i StandardizedDataset) i defineix l'estructura comuna.

Atributs:

- **int dim:** Dimensió dels vectors d'entrada.
- **List<Record> data:** Llista que emmagatzema tots els registres del dataset.

Constructor:

- **Dataset(int d):** Inicialitza el dataset i crea una llista buida de registres.

Mètodes concrets:

- **getDim():** Retorna la dimensió del dataset.
- **getData():** Retorna la llista completa de registres.
- **addRecord(Vector in, double out):** Crea un nou Record i l'afegeix a la llista de dades.

- **toString():** Retorna una representació en format text del dataset amb la seva dimensió, nombre de registres i contingut.

Mètodes abstractes:

- **transform(Record r):** Transforma un registre segons el tipus de dataset. Les subclasses han d'implementar aquest mètode per definir com es transformen les dades.
- **output(double transformedOutput):** Aplica la transformació inversa a un valor de sortida. Permet convertir prediccions de l'espai transformat a l'espai original.

2.2. Classe RawDataset

Aquesta classe representa un dataset amb dades en el seu espai original, sense cap transformació.

Constructor:

- **RawDataset(int d):** Crida al constructor de la superclasse Dataset.

Mètodes:

- **meanInput():** Calcula i retorna el vector mitjà de tots els vectors d'entrada del dataset.
- **stdInput():** Calcula i retorna la desviació estàndard per a cada component dels vectors d'entrada.
- **meanOutput():** Calcula i retorna la mitjana de tots els valors de sortida del dataset.
- **stdOutput():** Calcula i retorna la desviació estàndard dels valors de sortida utilitzant la fórmula de la desviació estàndard.
- **standardize():** Crea un nou StandardizedDataset a partir del RawDataset actual. Primer calcula les mitjanes i desviacions estàndard de les entrades i sortides, crea la instància de StandardizedDataset amb aquests paràmetres, transforma tots els registres i els afegeix al nou dataset estandarditzat. Retorna el StandardizedDataset resultant.

Implementació dels mètodes abstractes:

- **transform(Record r):** Retorna el mateix registre sense modificacions, ja que RawDataset no aplica cap transformació.
- **output(double transformedOutput):** Retorna el mateix valor sense modificacions.

2.3. Classe StandardizedDataset

Aquesta classe representa un dataset amb dades estandarditzades (mitjana 0 i desviació estàndard 1).

Atributs:

- **Vector mi**: Mitjana dels vectors d'entrada de l'espai original (μ_{in}).
- **Vector si**: Desviació estàndard dels vectors d'entrada de l'espai original (σ_{in}).
- **double mo**: Mitjana dels valors de sortida de l'espai original (μ_{out}).
- **double so**: Desviació estàndard dels valors de sortida de l'espai original (σ_{out}).

Constructor:

- **StandardizedDataset(int d, Vector mi, Vector si, double mo, double so)**: Inicialitza el dataset amb la dimensió especificada i emmagatzema els paràmetres de normalització necessaris per a les transformacions.

Implementació dels mètodes abstractes:

- **transform(Record r)**: Transforma un registre de l'espai original a l'espai estandarditzat aplicant les fórmules:

$$x'_i = \frac{x_i - \mu_x}{\sigma_x}, \quad y' = \frac{y - \mu_y}{\sigma_y}$$

Retorna un nou Record amb els valors estandarditzats.

- **output(double transformedOutput)**: Aplica la transformació inversa per convertir un valor de sortida de l'espai estandarditzat a l'espai original utilitzant la fórmula: $y = v * \sigma_{out} + \mu_{out}$, on v és el valor estandarditzat. Això permet que les prediccions del model entrenat amb dades estandarditzades es puguin interpretar en l'espai original.

2.4. Classe Algorithm

Aquesta classe abstracta proporciona l'estructura base per a tots els algorismes d'aprenentatge supervisat.

Atributs:

- **double learningRate**: Taxa d'aprenentatge (α) protegida (protected) que controla la mida dels passos durant l'optimització. És compartida per totes les subclasses.
- **int maxIterations**: Nombre màxim d'iteracions permeses (fixat a 10000) per evitar bucles infinits.

Constructor:

- **Algorithm(double lr)**: Inicialitza la taxa d'aprenentatge amb el valor proporcionat.

Mètode abstracte:

- **solve(Dataset ds)**: Mètode abstracte que ha de ser implementat per cada subclasse per entrenar i retornar un model donat un dataset.

Mètode concret:

- **gradient(Record r, Model m):** Calcula el gradient de la funció de pèrdua per a una única mostra. Utilitza la fórmula: $\nabla L = x * (h_{\theta}(x) - y)$. Aquest mètode és reutilitzat tant per GradientDescent com per StochasticGradientDescent.

2.5. Classe GradientDescent

Implementa el descens de gradient complet (batch gradient descent), que utilitza tot el dataset en cada iteració.

Atributs:

- **double stoppingCriterion:** Criteri d'aturada específic d'aquesta classe. L'algoritme s'atura quan la norma del gradient és inferior a aquest valor.

Constructor:

- **GradientDescent(double lr, double sc):** Crida al constructor de la superclasse amb la taxa d'aprenentatge i inicialitza el criteri d'aturada.

Mètodes:

- **fullGradient(Dataset ds, Model m):** Calcula el gradient complet utilitzant tots els registres del dataset. Amb el gradient (que crida de la classe abstracte) de cada registre obté la mitjana.
- **solve(Dataset ds):** Implementa el bucle principal del descens de gradient. Inicialitza un model nou amb tots els paràmetres a 0, calcula el gradient inicial i itera aplicant l'actualització $\theta = \theta - \alpha * \nabla L(\theta)$ mentre la norma del gradient sigui superior al criteri d'aturada i no s'hagi arribat al màxim d'iteracions. Al final mostra el nombre d'iteracions realitzades i la norma final del gradient, i retorna el model entrenat.

2.6. Classe StochasticGradientDescent

Implementa el descens de gradient estocàstic (SGD), que utilitza minibatches aleatoris per actualitzar el model.

Atributs:

- **int batchSize:** Mida del minibatch (k), és a dir, el nombre de mostres utilitzades en cada iteració per calcular el gradient.
- **int iterations:** Nombre total d'iteracions (èpoques) que l'algoritme realitzarà.
- **Random random:** Generador de números aleatoris utilitzat per barrejar les dades i seleccionar minibatches aleatoris.

Constructor:

- **StochasticGradientDescent(double lr, int bs, int it):** Crida al constructor de la superclasse amb la taxa d'aprenentatge i inicialitza la mida del batch, el nombre d'iteracions i el generador aleatori.

Mètodes:

- **stochasticGradient(Dataset ds, Model m):** Calcula el gradient estocàstic utilitzant un minibatch aleatori. Primer obté la llista completa de dades i la barreja utilitzant `Collections.shuffle()` per garantir aleatorietat. Determina la mida real del batch com el mínim entre `batchSize` i la mida total del dataset. Crea un minibatch seleccionant entre els primers `actualBatchSize` elements de la llista barrejada. Calcula el gradient per a cada mostra del minibatch utilitzant el mètode `gradient()` heretat. A continuació, divideix el gradient pel nombre de mostres del minibatch i multiplica per 2 (factor que prové de la derivada de l'Error Quadràtic Mitjà, MSE). Retorna el stochastic gradient resultant.
- **solve(Dataset ds):** Implementa el bucle principal del SGD. Inicialitza un model nou amb tots els paràmetres a 0 i itera un nombre fix de vegades determinat per l'atribut `iterations`. En cada iteració calcula un gradient estocàstic utilitzant `stochasticGradient()` i actualitza els paràmetres del model amb $\theta = \theta - \alpha * g$. Al finalitzar, mostra per pantalla el nombre d'iteracions completades i retorna el model entrenat.

2.7. Classe SupervisedLearner

Aquesta classe manté la mateixa estructura que en el Lab 3, però ajustat per gestionar transformacions en datasets estandaritzats.

Atributs:

- **Algorithm algorithm:** Algorisme d'aprenentatge utilitzat per entrenar el model.
- **Dataset dataset:** Conjunt de dades d'entrenament amb els vectors d'entrada i sortides reals.
- **Model model:** Model entrenat, inicialment null fins que s'executa el mètode `solve()`.

Constructor:

- **SupervisedLearner(Algorithm a, Dataset d):** Inicialitza el SupervisedLearner amb un algorisme i un dataset. El model es manté null fins que s'entrena.

Mètodes:

- **solve():** Delega l'entrenament del model a l'algorisme cridant el mètode `solve()` d'`Algorithm`, i emmagatzema el model resultant.
- **predict(Vector v):** Rep un vector d'entrada (sense augmentar), l'augmenta afegint el bias com a primer element, i retorna la predicció del model entrenat utilitzant el mètode `predict()` de `Model`. A més, aplica la transformació inversa mitjançant `dataset.output()` si el dataset és estandaritzat, retornant el valor de sortida en l'espai original.

- **toString():** Retorna una representació en format text del SupervisedLearner.

2.8. Classe TestLearner

Conté el mètode main() amb 9 tests que proven totes les funcionalitats implementades:

TEST 1 - RawDataset - Estadístiques: Crea un RawDataset amb 5 registres de dimensió 2 i verifica el càlcul correcte de mitjanes i desviacions estàndard. També comprova que els mètodes transform() i output() no modifiquin les dades.

TEST 2 - RawDataset + GradientDescent: Entrena un model utilitzant GradientDescent amb dades raw. Utilitza learning rate 0.01 i stopping criterion 0.001. Mostra el model entrenat i fa prediccions per verificar el funcionament.

TEST 3 - StandardizedDataset - Transformacions: Crea un StandardizedDataset a partir del RawDataset anterior i verifica que els registres estan correctament estandarditzats. Prova les transformacions i destransformacions.

TEST 4 - StandardizedDataset + GradientDescent: Entrena un model amb GradientDescent utilitzant dades estandarditzades. Observa que es pot utilitzar un learning rate més gran (0.1) perquè les dades estan normalitzades. Verifica que les prediccions es destransformen correctament a l'espai original.

TEST 5 - RawDataset + StochasticGradientDescent: Entrena un model utilitzant SGD amb dades raw. Utilitza learning rate 0.005, batch size 3 i 2000 iteracions. Compara els resultats amb GradientDescent.

TEST 6 - StandardizedDataset + StochasticGradientDescent: Entrena un model amb SGD utilitzant dades estandarditzades amb learning rate 0.05, batch size 3 i 2000 iteracions.

TEST 7 - Comparació de resultats: Compara les prediccions dels 4 models entrenats (GD raw, GD standardized, SGD raw, SGD standardized) per al mateix input, verificant que tots donen resultats similars.

TEST 8 - Dataset més gran: Crea un dataset amb 100 registres seguint la relació $y = 2x + 3$ amb soroll aleatori. Entrena models amb GD i SGD i compara els resultats, demostrant que SGD funciona bé amb datasets grans.

TEST 9 - Verificació de polimorfisme: Prova totes les combinacions possibles de Dataset (RawDataset i StandardizedDataset) amb Algorithm (GradientDescent i StochasticGradientDescent) utilitzant arrays i bucles, demostrant que el polimorfisme funciona correctament.

3. Conclusions

La implementació del Lab 4 s'ha completat amb èxit, incorporant conceptes com classes abstractes, herència, polimorfisme i refactorització de codi.

La refactorització de les classes Dataset i Algorithm per convertir-les en abstractes ha millorat significativament la modularitat i extensibilitat del codi. Aquesta decisió de disseny permet afegir nous tipus de datasets o algoritmes en el futur sense modificar el codi existent, seguint el principi Open/Closed de SOLID. La implementació del descens de gradient estocàstic (SGD) ha estat la part més complexa del laboratori. Ha calgut entendre com seleccionar minibatches aleatoris de forma eficient utilitzant `Collections.shuffle()`, i ajustar adequadament els hiperparàmetres (learning rate, batch size i nombre d'iteracions) per aconseguir una convergència correcta.

Hem observat que SGD requereix un learning rate lleugerament inferior al de GD, ja que el gradient estocàstic és menys precís i pot provocar oscil·lacions si el learning rate és massa gran. Tot i això, SGD pot ser molt més ràpid en datasets grans, ja que no necessita processar totes les mostres en cada iteració.

La modificació del mètode `predict()` en el `SupervisedLearner` per gestionar transformacions garanteix que el sistema funcioni correctament amb datasets estandarditzats.

Gràcies als mètodes abstractes `transform()` i `output()` definits a Dataset, el codi és genèric i funciona automàticament amb qualsevol tipus de dataset sense necessitat de condicionals ni comprovacions.